Formal Paper Specification: Hierarchical TOPOSCAN with RL-Based Multi-Hop Assignment

---

Section 1: Foundational Definitions

1.1 Road Network

Let $G = (V, E)$ be a directed road network graph where:

- V is the set of road nodes
- $E \subseteq V \times V$ is the set of directed road segments (links)
- $\ell: E \to \mathrm{R}^{+}$ is the physical length function on each link

The **road-network distance** between two nodes $u, v \in V$ is:

$$d_G(u, v) = \min_{\pi \in \Pi(u,v)} \sum_{e \in \pi} \ell(e)$$

where $\Pi(u, v)$ is the set of all directed paths from u to v in G.

1.2 Trajectory

A **trajectory** is an ordered sequence of directed links:

$$\tau = \langle e_1, e_2, \ldots, e_n \rangle, e_i \in E, \forall i : \ \mathrm{end}(e_i) = \mathrm{start}(e_{i+1})$$

The set of all observed trajectories is the **trip corpus** τ.

1.3 Trunk

A **trunk** T_k is an ordered sequence of directed links forming a valid contiguous path in G:

$$T_k = \langle e_1, e_2, \ldots, e_m \rangle, e_i \in E$$

with:

- **Access node:** $\alpha(T_k) = \text{start}(e_1)$
- **Egress node:** $\omega(T_k) = \text{end}(e_m)$
- **Interior nodes:** $I(T_k)$ = {tail(e_i) : i = 1,…, m-1}
- **Physical length:** $\text{L}(T_k) = \sum_{i=1}^{m} \text{l}(e_i)$
- **Node set:** P(T_k) = $\{\alpha(T_k)\} \cup \text{I}(T_k) \cup \{\omega(T_k)\}$

Containment: A trajectory τ contains trunk T_k (written $T_k \sqsubseteq \tau$) if and only if T_k appears as a contiguous directed subsequence of τ.

Section 2: Hierarchical TOPOSCAN Extraction

2.1 Layer Parameters

We define L extraction layers with strictly cascading parameters:

$$\text{Layer}\ell = 1, \ldots, L\text{: } (\alpha_\ell, \mu_\ell)$$

where:

- α_ℓ: minimum path density (Alpha threshold) for Layer ℓ
- μ_ℓ: minimum physical trunk length for Layer ℓ

satisfying the **cascade conditions**:

$$\alpha_1 > \alpha_2 > \cdots > \alpha_L \geq 1\ \mu_1 > \mu_2 > \cdots > \mu_L > 0$$

2.2 The Evidence Contribution Principle (Interpretation A)

A trajectory τ is said to **contribute evidence** to trunk T_k if and only if:

$$T_k \sqsubseteq \tau$$

i.e., the trajectory physically traverses the trunk.

After a layer is extracted, the set of contributing trajectories is removed from the active pool. This prevents the algorithm from re-discovering the same dominant corridor in subsequent layers. It does **not** imply assignment; it is purely an evidence-exhaustion mechanism.

2.3 Hierarchical Extraction Algorithm

Let $\tau_{(0)} = \tau$ (the full trip corpus).

For each layer $\ell = 1,2, \dots, L$:

Step 1 — Extraction:
Run TOPOSCAN on $\tau_{(\ell-1)}$ with parameters (α_ℓ, μ_ℓ) to produce the set of Layer-ℓ trunks:

$$\mathbb{T}_{(\ell)} = \text{TOPOSCAN}(\tau_{(\ell-1)}, \alpha_\ell, \mu_\ell)$$

Step 2 — Evidence Removal:
Remove all trajectories that contributed to any trunk in $\mathbb{T}^{(\ell)}$:

$$\tau_{(\ell)} = \tau_{(\ell-1)} \setminus \{t \in \tau_{(\ell-1)} : \exists T_k \in \mathbb{T}_{(\ell)}, T_k \sqsubseteq \tau\}$$

The **full hierarchical trunk set** after all L layers is:

$$\mathbb{T} = \bigcup_{\ell=1}^{L} \mathbb{T}^{(l)}$$

Each trunk $T_k \in \mathbb{T}$ carries a **layer label** $\lambda(T_k) \in \{1,\ldots, L\}$.

Section 3: Transfer Points and Transfer Feasibility

3.1 Transfer Point

Definition (Transfer Point):
A node $p \in V$ is a **transfer point** on trunk T_k if and only if:

$$p \in P(T_k)$$

That is, p physically lies on the trunk (either the access node, an interior intersection, or the egress node).

3.2 Transfer Feasibility

Definition (Feasible Hop):
A hop from trunk T_a to trunk T_b is **feasible** (written $T_a \rightsquigarrow T_b$) if and only if:

$$\exists p_a \in P(T_a), p_b \in P(T_b) \text{ such that } d_G(p_a, p_b) \leq \delta$$

Where $\delta = \delta_a + \delta_b$

Where δ is the **Detour Threshold**, a tunable parameter in meters.

Each layer has its own $\delta_i \in \{\delta_1, \delta_2, \ldots, \delta_L\}$ & $\delta_1 < \delta_2 < \cdots < \delta_L$

δ_ℓ represents the maximum transfer detour tolerated by vehicles operating on layer ℓ.

The **optimal transfer pair** for a hop $T_a \rightsquigarrow T_b$ is:

$$(\hat{p}_a, \hat{p}_b) = arg \min_{p_a \in \mathrm{P}(T_a), p_b \in \mathrm{P}(T_b)} d_G(p_a, p_b)$$

and the **hop cost** is:

$$c_{\mathrm{hop}}(T_a, T_b) = d_G(\hat{p}_a, \hat{p}_b)$$

Section 4: The Transfer Graph

4.1 Structure

The **Transfer Graph** $\mathrm{G_T} = (\mathrm{V},\mathrm{E})$ is defined as:

Nodes:

$$\mathrm{V} = \mathrm{T} = \{T_1, T_2, \dots, T_K\}$$

Each node is a trunk. Source (O) and sink (D) nodes representing the query trip's origin and destination region are added as virtual nodes.

Edges:

$$\mathcal{E} = \{(T_a, T_b) : T_a \rightsquigarrow T_b\}$$

An edge exists if and only if the hop is physically feasible.

$$\varepsilon = \{(O, T_k) \colon \exists x \in P(T_k), d_G(O, x) \leq \delta_{\lambda(T_k)}\}$$

$$\varepsilon = \{(T_k, D): \exists x \ \in P(T_k), d_G(x, D) \leq \delta_{\lambda(T_k)}\}$$

Edge Weight:

$$w(T_a, T_b) = c_{\text{hop}}(T_a, T_b) - \lambda \cdot \sigma(T_a, T_b)$$

where:

- c_{hop}: physical detour cost (road-network distance)
- $\sigma(T_a, T_b) \in [0,1]$: NLP semantic similarity (cosine similarity of trunk embeddings)
- $\lambda > 0$: scaling parameter

Section 5: NLP Trajectory Embeddings

5.1 Traj2Vec Transformation

Each directed link $e = (u, v) \in E$ is treated as a discrete token $w_e =$ `"u_v"`.

Each trajectory $\tau = \langle e_1, \dots, e_n \rangle$ is treated as a sentence:

$$\text{sent}(\tau) = [w_{e_1}, w_{e_2}, \dots, w_{e_n}]$$

A **Word2Vec** model is trained on the corpus $\{\text{sent}(t) : t \in \{ \tau \}\}$ to produce a vector embedding function:

$$\phi: E \to \mathrm{R}^d$$

5.2 Trunk Embedding

The embedding of trunk T_k is the mean-pooled vector of its link embeddings:

$$\Phi(T_k) = \frac{1}{|T_k|} \sum_{e \in T_k} \phi(e) \in \mathrm{R}^d$$

5.3 Target Context Embedding

$$\Phi(D) = \frac{1}{|\mathcal{N}_D|} \sum_{T_k \in \mathcal{N}_D} \Phi(T_k)$$

Where

$$\mathcal{N}_D = \{T_k \in T : d_G(\omega(T_k), D) \leq \delta\}.$$

5.4 Semantic Similarity

$$\sigma(T_a, T_b) = \frac{\Phi(T_a) \cdot \Phi(T_b)}{\|\Phi(T_a)\| \|\Phi(T_b)\|}$$

Section 6: Hierarchical Violation Penalty

6.1 Motivation

Hierarchical layers are descriptive rather than prescriptive. Consequently, transitions between any pair of physically feasible trunks are permitted regardless of layer differences.

However, a route may occasionally enter a trunk and exit it almost immediately at a nearby transfer location. Such behaviour provides negligible progression through the network and

often indicates an unnecessary transfer.
To discourage this behaviour, a Transfer Oscillation Penalty is introduced.

6.2 Transfer Entry and Exit Points

Consider a sequence of consecutive hops

$$T_a \rightarrow T_b \rightarrow T_c.$$

Let

$$x_{ab}$$

denote the optimal transfer point on trunk T_b used when transferring from T_a to T_b.

Similarly, let

$$x_{bc}$$

denote the optimal transfer point on trunk T_b used when transferring from T_b to T_c.

The progression made while traversing trunk T_b is measured by the road-network distance between the entry and exit transfer points:

$$\delta(T_a, T_b, T_c) = d_G(x_{ab}, x_{bc}).$$

6.3 Oscillation Penalty Definition

Let

$$\tau_m$$

be the minimum progression threshold.

A transfer oscillation occurs when the distance travelled between the entry and exit transfer points on the intermediate trunk is less than τ_m.

The oscillation penalty is therefore defined as

$$h(T_a, T_b, T_c) = \begin{cases} 1, & \delta(T_a, T_b, T_c) < \tau_m \\ 0, & \text{otherwise.} \end{cases}$$

Equivalently,

$$h(T_a, T_b, T_c) = \begin{cases} 1, & d_G(x_{ab}, x_{bc}) < \tau_m \\ 0, & d_G(x_{ab}, x_{bc}) \geq \tau_m. \end{cases}$$

6.4 Interpretation

The penalty is independent of hierarchical layer differences.

Therefore, transitions such as

$$L_3 \rightarrow L_1 \rightarrow L_3$$

or

$$L_1 \rightarrow L_3 \rightarrow L_1$$

are not penalized solely because of the layers involved.

A penalty is incurred only when the entry and exit transfer points on the intermediate trunk are spatially close, indicating that the trunk was used without meaningful progression through the network.

Section 7: RL Formulation

7.1 State Space

At each decision step t, the RL state is:

$$s_t = [\Phi(T_t), \Phi_D,\ \lambda(T_t),\ k_t]$$

where:

- $\Phi(T_t) \in \mathrm{R}^d$: NLP embedding of the current trunk
- $\Phi_D \in \mathrm{R}^d$: NLP embedding of the target destination context
- $\lambda\ (T_t) \in \{1, 2, 3\}$: layer tier of the current trunk (encoded as one-hot)
- $k_t \in \mathrm{N}$: current hop count (encodes remaining budget)

State dimension: $2d + 3 + 1$.

7.2 Action Space

At step t, the agent is on trunk T_t. The **valid action set** is:

$$\mathcal{A}(T_t) = \{T_b \in \mathbb{T} : T_t \rightsquigarrow T_b\}$$

This set is derived directly from the edges of the Transfer Graph $\mathrm{G_T}$.

The action space is dynamic: its size varies by the current trunk's position in the network.

Masking: Physically infeasible hops are masked with $-\infty$ in the logit layer before Softmax, ensuring zero probability.

7.3 Reward Function

At each hop step t, the agent receives:

$$r_t = r_{\text{semantic}} + r_{\text{efficiency}} + r_{\text{oscillation}} + r_{\text{goal}}$$

Semantic Reward (aligning with destination): $r_{\text{semantic}} = \lambda_s \cdot \sigma(T_t, T_D)$

where T_D is a virtual trunk embedding constructed from the destination context.

Efficiency Penalty (discouraging unnecessary hops): $r_{\text{efficiency}} = -c_{\text{hop}}(T_{t-1}, T_t)$

Oscillation Penalty: $r_{oscillation} = -\rho h(T_{t-2}, T_{t-1}, T_t)$

Where

$$h(T_{t-2}, T_{t-1}, T_t) = \begin{cases} 1, & d_G(x_{t-2,t-1},\ x_{t-1,t}) < \tau_m \\ 0, & otherwise \end{cases}$$

Goal Reward (if egress from T_t is within δ of destination D):

$$r_{\text{goal}} = \begin{cases} R_{\text{success}}, & \text{if } d_G(\omega(T_t), D) \le \delta \\ -R_{\text{fail}}, & \text{if } t = t_{max} \text{ and goal not reached} \\ 0, & otherwise \end{cases}$$
